

# Test Case Prioritization Based on Neural Networks Classification

Cristina-Maria Tiutin  
cristina.tiutin@stud.ubbcluj.ro  
Babes-Bolyai University  
Faculty of Mathematics and Computer Science  
Romania

Andreea Vescan  
andreea.vescan@ubbcluj.ro  
Babes-Bolyai University  
Faculty of Mathematics and Computer Science  
Romania

## ABSTRACT

Regression testing focuses on validating modified software, in order to detect if new errors were added into previously tested code and to provide confidence that modifications are correct. An approach that involves running all test cases would be time-consuming, however, test case prioritization plans an execution order of the test cases as an attempt to achieve the regression testing goals early in the testing phase.

In this paper, we propose a Test Case Prioritization based on Neural Networks Classification (TCP-NNC) approach to be further used in the test case prioritization strategy. The proposed approach incorporates among other factors, the associations between requirements, tests and discovered faults, based on which an artificial neural network is trained, in order to be able to predict priorities for new test cases. The proposal is evaluated through experiments designed on both a real and a synthetic dataset, considering two different sets of features with different neural network architectures. The metrics observed include accuracy, precision and recall, while their results imply that the proposed method is feasible and effective. Among the proposed models, the one with Adam optimizer and three-layered architecture is the best obtained. Statistical tests are also used to compare various proposed models from various perspectives: NN architecture, optimizer, number of used features, used dataset and validation method.

## CCS CONCEPTS

• **Software and its engineering** → **Maintaining software; Software evolution**; • **Computing methodologies** → **Neural networks**.

## KEYWORDS

Regression testing, Test case prioritization, Neural networks, Test case classification, Requirements dependencies, Metrics

### ACM Reference Format:

Cristina-Maria Tiutin and Andreea Vescan. 2022. Test Case Prioritization Based on Neural Networks Classification. In *Proceedings of the 2nd ACM International Workshop on AI and Software Testing/Analysis (AISTA '22)*, July 18, 2022, Virtual, South Korea. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3536168.3543300>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

AISTA '22, July 18, 2022, Virtual, South Korea

© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.  
ACM ISBN 978-1-4503-9387-4/22/07...\$15.00  
<https://doi.org/10.1145/3536168.3543300>

## 1 INTRODUCTION

Regression testing [2] is an essential process that most of the software systems go through a various number of times within their development and maintenance activities. Its target focuses on validation of modified software, aiming to detect if new errors were added into previously tested code and to provide confidence that modifications are correct. However, the challenge which arises in regression testing refers to the fact that test suites are prone to become so large that running all test cases would be time consuming. In the cases when regression tests do not manage to finish within a reasonable amount of time, the development process is disrupted. There exist many approaches dealing with the regression testing problem [4], ranging from test suite minimization to test case selection and test case prioritization.

Test Case Prioritization (TCP) is a technique that tries to optimize which test cases should be executed first, setting them up in a certain order such that faults are detected as soon as possible. Test case selection and prioritization methods undertake the costly execution time of regression testing by choosing and prioritizing test cases which are sufficient to test new changes and are able to detect faults early in the testing phase.

Recently, there has been observed an interest towards machine learning-based approaches in TCP. The basic workflow of such a technique starts with creating a set of training data, comprising of various features of the test cases. Each test case is assigned a label corresponding to a priority class, based on some criteria. A machine learning algorithm can then be trained on this dataset, in order to classify new test cases. The general idea is that test cases predicted with highest priority disclose the most important faults early in the testing phase.

The aim of this paper is conducting a replication of [5], thus to investigate a test case classification approach using supervised machine learning methods. We also propose other features to be incorporated and investigate new architectures of the neural network models. Our goal is to set an order of test cases, based on a given training dataset, which corresponds to the intuition of a tester, while priorities are assigned to new test cases. To meet this objective, an artificial neural network (ANN) based classifier is built, using supervised training data. ANNs have the capability of understanding a highly complex system based on its behavior. Test cases also represent a system's behavior in terms of various execution scenarios, making it a fair idea to use ANNs to prioritize test cases. In our approach we use both *static* data (requirements to test cases relation, requirements dependencies) and *dynamic* data (cost of execution of test cases, faults discovered).

The paper by Vescan et al. [14] proposes the TCP formalization that considers not only test requirements but also functional requirements and dependencies between the functional requirements.

Our approach builds on this proposal, considering new metrics as features, introducing a new neural network approach for test case classification using the *grep* project [1] and a new synthetic dataset constructed based on *grep* project samples.

The contributions of this paper are the followings:

- An approach for test case classification (to further be used for test case prioritization) based on various factors, including the associations between requirements, tests and discovered faults. Thus, new metrics are considered as criteria (not only requirements as code coverage but requirements as features).
- An operationalization of our approach using ANN in order to obtain for new test cases their assigned execution priorities.
- Experiments with models incorporating various features and with model designs with various architectures and optimizer.
- A set of experiments designed for both real and synthetic data, to prove the efficiency of our approach regarding fault detection rate and prediction accuracy.

The rest of the paper is organised as follows: in Section 2 we discuss background concepts regarding test case prioritization. Section 3 briefly reviews the existing related work. Section 4 presents the overview of the approach. Section 5 outlines our proposed approach, presenting an overview, the used dataset, proposed models and also information on conducted experiments. We interpret the results in Section 6. We present the limitations of the study in Section 7. Section 7 concludes the paper with a discussion of future work.

## 2 TEST CASE PRIORITIZATION BACKGROUND

Regression testing [4], with all its existing strategies, plays a significant part in identifying and fixing faults, after changes are incorporated in the system. Test case prioritization is one such strategy, which intends to provide important information about the system under test, concerning the best order of test cases that may identify existing faults.

Graves defines TCP in his paper [4], as follows.

**Test case prioritization [4]:** Given a test suite,  $T$ , the set of permutations of  $T$ ,  $PT$ ; a function from  $PT$  to real numbers,  $f$ .

*Problem:* to find  $T' \in PT$  such that

$$(\forall T'')(T'' \in PT)(T'' \neq T')[f(T') \geq f(T'')] \quad (1)$$

The function  $f$  assigns a real value to a permutation of  $T$  according to the test adequacy of the particular permutation.

The ideal order would be one that reveals faults the earliest, as measured by the Average Percentage Faults Detected (APFD) metric. Due to the fact that fault detection is not known in advance, regression testing approaches use a variety of code coverage criteria. A number of approaches with different coverage measurements are analyzed by Rothermel et al. [10], showing that the rate of fault detection can be improved by coverage prioritisation [16].

In our proposal, we build on top of the representation from the work of Vescan et al. [14] that formalizes the TCP problem by adding the relation between requirements and test cases, and dependencies between requirements. Thus, based on the requirements set, the requirement dependencies table and the test cases set, we compute

the number of modified requirements and the number of dependent requirements covered by each test case.

## 3 RELATED WORK

In this section, we intend to briefly survey existing test case prioritization approaches with focus on soft computing techniques and related work in this area. This section is meant to position our work with respect to the existing research.

A systematic literature review (SLR) [6] was conducted with the aim to indicate the state-of-the-art and to discover the on-going scientific research on TCP-related algorithms that apply one of the various soft computing approaches.

The search was conducted on two databases and one indexing system. We defined several appropriate keywords (regression testing, test case prioritization, fault detection, code coverage, defect prediction, soft computing, neural networks, particle swarm optimisation, machine learning, swarm intelligence), based on the aspects we want to focus our research on. The search results were merged together, having duplicates and entries which were not research papers removed. Since soft computing techniques are more recent approaches, we concentrated on papers published starting with the year 2000. Thirty-two studies were found to be relevant for our work. In what follows, we will detail some of them.

A solution relying on Particle Swarm Optimisation (PSO) is given in the work of Tyagi et al. [13]. The test case prioritization problem is addressed here in three phases. The first one focuses on saving resources and time by the removal of redundant test cases, using simple matrix operations. Second phase includes Multi Objective Particle Swarm Optimization (MOPSO) for selecting test cases based on fault coverage and execution time. The selected test cases should have the ability to cover all faults, at the minimum execution time. The actual test case prioritization is performed in the third phase, considering both the test suite reduction from first step and the test case selection from the second step, based on fault coverage and execution time of test cases.

One paper based on Ant Colony Optimisation (ACO) [12] focuses on regression test prioritization technique conditioned by time constraint environment. The priority is set according to the chosen criteria, which may be to increase rate of fault detection, to achieve maximum code coverage or to cover important features earlier. Time constraint test case prioritization problem can be solved through ACO. The preliminary results are encouraging: using APFD, prioritization achieved using ACO leads to an ordering that is comparable to optimum ordering. ACO is strong and robust as it involves positive feedback and parallel computations; hence, it can lead to better solutions in optimum time.

The work of Vescan et al. [14] represents a review of several algorithms. The most efficient test schedules appear to be identified when a Greedy algorithm or a Genetic Algorithm is used. Both functional and non-functional requirements are included with the quality assessment of a test order. The experiments conducted in this paper looked at different criteria combinations, such as faults, costs, number of test cases, and the reached outcome was that for all the criteria, Genetic Algorithm exceeds the Greedy algorithm.

Using machine learning as soft computing technique, the proposal of Lachmann et al. [7] is a novel technique that allows for

automatic test case prioritization in system-level testing of test cases written in natural language. It aims to emulate test expert knowledge and to reuse it for an automatic prioritization of test cases. The approach considers black-box meta-data, such as test case history, as well as natural language test case descriptions for prioritization. Supervised Machine Learning (ML) and Natural Language Processing (NLP) are used to rank test cases.

Another is provided in paper [5]. The authors propose a version specific test case prioritization based on three factors, with the intention to improve fault detection rate and cost effectiveness. A machine learning-based supervised approach is used. An ANN classifier is built and trained in order to assign priorities to new test cases. The effectiveness of the approach in terms of fault detection rate and prediction accuracy is validated through two phases of empirical studies.

Our approach builds on top of the [14] TCP formal definition and uses the neural network classification approach from [5] with additional improvements.

#### 4 TEST CASE PRIORITIZATION BASED ON CLASSIFICATION

Regression testing is approached with various strategies, from retest all to test suite minimization (TSM), test case selection (TCS) and test case prioritization (TCP). Figure 1 graphically depicts the two approaches that we are going to use in this paper: the Test Case Prioritization that uses the classification for each test case to further order the test suite based on this classification, and the *retest all* approach using random execution of the test cases in the test suite. As in definition from Section 2, given  $T = (t_1, t_2, \dots, t_n)$ , *retest all* with random strategy means generating randomly a permutation of  $T$ . The neural network classification based TCP means using the classification of the test cases to order them from most critical to less critical.

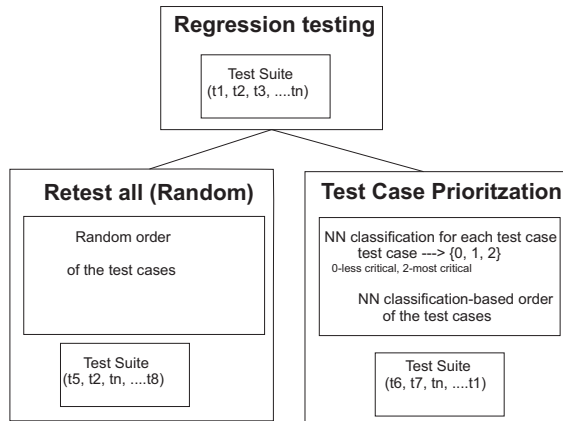


Figure 1: Regression testing strategies used in this paper: Test Case Prioritization that uses the neural network classification and the Retest all approach

#### 5 NEURAL NETWORK TEST CASE CLASSIFICATION APPROACH

In the next section we will present the details of the proposed approach, emphasizing the various employed Neural Network models. The dataset along with the design of experiments and the analysis methods are also described.

The overall framework of our *Test Case Prioritization based on Neural Networks Classification (TCP-NNC)* approach contains three phases, i.e., data preparation, training and deployment/validation. Figure 2 graphically depicts the process, following the neural network - based approach methodology.

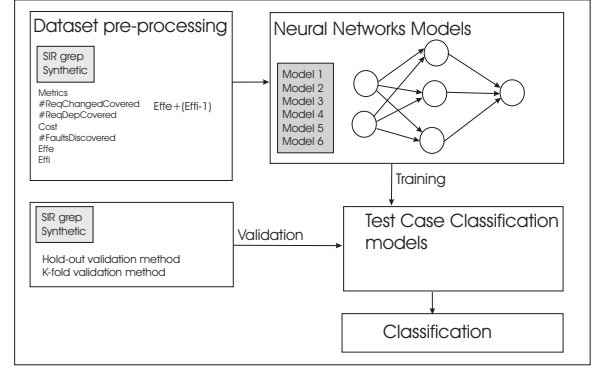


Figure 2: Neural Network Test Case Classification approach for Test Case Prioritization Strategy

##### 5.1 Features

Using the information from the TCP formalization in [15] (tests - faults information, tests - requirements information, requirements dependencies, test case execution cost as time, effectiveness of a test case and efficiency of a test case) and also the target vector defined as in paper [5] (modified modules and modified requirements), we compose our feature collection.

As shown in Figure 2 the feature vector uses the following metrics computed for each test cases, values that are based on the requirement set, the requirement dependencies table, test case set and faults set:

- *#ReqChangedCovered* - number of modified requirements covered by corresponding test case;
- *#ReqDepCovered* - number of dependent requirements covered by corresponding test case;
- *cost* - normalised cost of the corresponding test case;
- *#FaultsDiscovered* - number of faults discovered by corresponding test case;
- *Effe* - defined as in [15];
- *Effi* - defined as in [15].

For the target vector we have used the same approach as in paper [5], incorporating the information regarding the modified modules and the modified requirements in our dataset. Thus, the value used for each test case for the labeling is computed as follows:  $Effe + (Effi - 1)$ . Based on the obtained values we have established three classes of test cases: 143 samples with label  $\leq 0.9980$ , 183

samples with values  $> 0.9980$  and  $\leq 0.9989$ , and 144 samples with values  $> 0.9990$ . We decided to use three classes as specified in [5], with the same remark regarding balance distributions of samples. Future work could investigate how to establish the threshold and the number of classes for labels.

In a first phase, we structured our approach rather as a replication study of the work from [5]. We implemented the multi-classification approach described in the paper, by following the mentioned steps, which include: creating a target vector with balanced distribution of samples by labeling the test cases with priority values from 0 to 2, building an ANN classifier with the reported architecture and pre-processing the training sets by encoding classes into a binary class matrix and scaling and standardizing the input feature vectors.

However, based on the neural network structure described in the paper, we also created our own experiments with another architectures and optimizer function. Regarding the feature vector that is created for a test case, we kept *the number of changed requirements covered* and *number of discovered faults* as factors considered for prioritization, but also added other features such as: *number of dependent requirements covered*, *cost*, *effectiveness* and *time efficiency*. In addition, we provide different results, based on more metrics, some of which are used in the original paper as well.

## 5.2 Neural Networks Models

An ANN [11] consists of an interconnected network of neurons, having a three-fold organization: an input layer, one or more hidden layers and an output layer. Synaptic weights transmit the input signal from the input layer to the first hidden layer. Activation functions determine whether the value should be passed on to the next layer based on the weighted summation of the inputs in the hidden layer neurons.

When building the model, several parameters need to be taken into account, such as: the number of neurons in the input layer, the number of hidden layers, the number of neurons in each hidden layer, the number of neurons in the output layer, activation function, the number of epochs. The number of input units corresponds to the inputs of the subject programs. The output layer contains three units, denoting the three priority classes. Relu activation function is used for hidden layers, while sigmoid function is used for the output layer. These parameters correspond to those used in [5].

We considered five more different models: (i) one with two hidden layers, each containing 20 units (as in the paper [5] but with Adam optimizer, Model 2); (ii) two with two hidden layers, each containing 30 units (one model with the same optimizer as in paper [5], Model 3, and one with Adam optimizer, Model 4); (iii) one with three hidden layers, each containing 30 units (Model 5 with the same optimizer as in paper [5] and Model 6 with Adam optimizer). In this research, we applied the stochastic gradient descent and Adam optimization algorithms. These models are described in Table 1.

Thus, to summarize the proposal, we have:

**Architecture Model - as proposed in [5].** The Neural Network architecture used in paper [5] is formed of two hidden layers each containing 20 nodes. The output layer consists of three units based on the three priority classes. For hidden layers, the activation

**Table 1: Architecture of the Neural Networks Models**

| Model   | Layers | Hidden neurons | Optimizer |
|---------|--------|----------------|-----------|
| Model 1 | 2      | 40             | sgd       |
| Model 2 |        |                | Adam      |
| Model 3 | 2      | 60             | sgd       |
| Model 4 |        |                | Adam      |
| Model 5 | 3      | 60             | sgd       |
| Model 6 |        |                | Adam      |

function Relu is applied, while sigmoid function is used for the output layer. This model is *Model 1* in Table 1.

**Architecture Models - Our Proposals.** We proposed other Neural Network architectures that uses Adam optimized and with various layers and different numbers of hidden neurons. The proposed models in Table 1 are from *Model 2* to *Model 6*.

## 5.3 Dataset

The literature on regression testing techniques [17], [8] reveals extended use of specific software repositories. This case study draws data from Software-artifact Infrastructure Repository (SIR), that contains software used in various types of testing experiments [1].

**5.3.1 Real Dataset - grep Object.** From the available software programs, named *objects* on SIR [1], **grep** object was chosen. In addition to providing a high number of test cases, it met the prerequisite of having the test information in the required format. The test suites were created for the base versions of the grep program, serving as regression suites for subsequent versions. The performance of testing techniques is evaluated with respect to detection of regression faults, that is, faults created in a program version as a result of the modifications that produced that version. There are 470 test cases per test suite and 75 requirements (denoted from  $r_1$  to  $r_{75}$ ) for **grep**. The changes from version  $v_0$  to  $v_1$  contain the followings: i) in the new version of the program, three requirements were modified:  $r_4$ ,  $r_{58}$ ,  $r_{68}$ ; ii) between requirements there are ten dependencies:  $r_3$ ,  $r_{45}$ ,  $r_{46}$ ,  $r_{49}$ ,  $r_{50}$ ,  $r_{51}$ ,  $r_{52}$  are influenced by  $r_{58}$ , and requirements  $r_{60}$ ,  $r_{62}$ ,  $r_{64}$  are influenced by  $r_{68}$ .

**5.3.2 Synthetic Dataset - grep - Based Generated Samples.** Using the methodology of generating instance spaces for machine learning classification in paper [9], we have constructed a new dataset starting from the **grep** object dataset, augmenting the instances with additional 500 samples. The used method generates test instances by evolving datasets to lie at target locations in the instance space. We briefly explain how the synthetic dataset is generated by fitting Gaussian mixture models.

The most common method of creating artificial test instances involves selecting and sampling an arbitrary probability distribution. This approach, however, lacks control, since there is no guarantee that the resulting dataset will have specific features. We used a Gaussian Mixture Model (GMM) [9] to generate instances with a desired target vector of features, until the mean squared error (MSE) between the target and the feature vector of a sample from the GMM is zero. Hellinger distance is a metric used to measure the difference between two probability distributions. We used it to check the amount of resemblance between the distributions of



feature values of original and synthetic data. Moreover, the dataset was standardized, as the Hellinger Distance appeared to improve for couple of features.

## 5.4 Experimental Setup

The designed experiments considered all the above models using both the **grep** object dataset [1] and the synthetic dataset built by generating additional samples as described in Section 5.3. We have also considered experiments with models that contain only part of the features (*#ReqChangedCovered* - number of modified requirements covered and *#ReqDepCovered* - number of dependent requirements covered) and models that contain all the previous specified features.

Figure 3 outlines the various dimensions that we considered in our experiments:

- various architectures (6 NN models),
- various set of features (partial - requirements covered and requirements dependency covered, all),
- dataset (real grep, synthetic based on grep),
- validation (hold-out, k-fold validation).

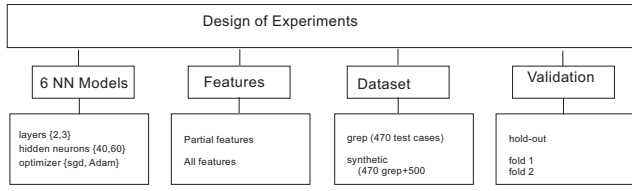


Figure 3: Design of experiments. Overview.

To partition the dataset into training and validation sets, two approaches were used: 80%-20% train-test split method and k-fold cross-validation method. Since the model can train on multiple train-test splits, cross validation is usually the preferred method. This will provide a better indication of how well the model will work on unseen data. Holdout, on the other hand, relies on just one train-test split. In that case, the holdout method score will depend on how the data is divided between train and test sets. More concrete details about how we apply them in our approach are provided in Section ??.

## 5.5 Analysis Methods

The conducted analysis of the proposed ANN model is performed from three perspectives: using the model performance metrics, statistical tests and APFD metric [10].

We used the same performance metrics [5]:

- Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- Precision

$$Precision = \frac{TP}{TP + FP}$$

- Recall

$$Recall = \frac{TP}{TP + FN}$$

- F1

$$F1 = \frac{2 * Precision * Recall}{Precision + Recall} = \frac{2 * TP}{2 * TP + FP + FN}$$

The APFD metric [10] is defined as follows (n is the number of test cases, m is the number of faults, and  $TF_f$  represents the position of the test case in the prioritized test suite that detects the fault f). The higher the value of the APFD score, the better the fault detection.

$$APFD = 1 - \frac{TF_1 + TF_2 + \dots + TF_m}{m \times n} + \frac{1}{2 \times n} \quad (2)$$

To compare the proposed models we have used the Wilcoxon Signed Rank Test [3] that is a nonparametric (normality of the data is not satisfied) procedure employed in hypothesis testing situations, involving a design with two samples. The Wilcoxon signed ranks test has two hypotheses:

- (1) Null hypothesis  $H_0$ : The median difference is zero.
- (2) Research hypothesis  $H_1$ : The median difference is not zero,  $\alpha = 0.05$ .

The applied statistical tests used the accuracy metric and were conducted to find out if there are any differences between the models considering various aspects. First, we compared the models that used partial features with those that use all considered features. Secondly, we studied if there are any differences within the models with the same architecture but different optimizer. Thirdly, we investigated if there are differences within the models that used the same optimizer but varying the architecture.

An overview of the used analysis methods and metrics is provided in Figure 4:

- metrics used to analyze the Neural Network models,
- statistical analysis (Wilcoxon Signed Rank Test) to investigate the difference between the proposed modes regarding various factors,
- APFD metric [10] to compare our NN classification-based TCP approach with a based line, i.e. retest all (random approach).

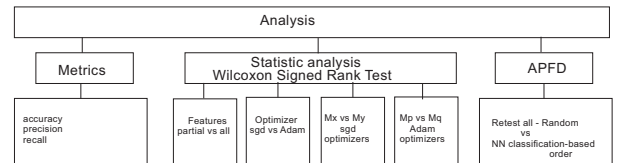


Figure 4: Analysis methods. Overview.

## 6 RESULTS

The obtained results for the performed experiments are outlined in this section. First, the results of experiments for models considering partial features and for models considering all features. Then, the results of the statistical tests comparing the models between them: models with partial features versus models with all features, models with same architecture but different optimizers, and models with same optimizer but different architectures. Eventually, results comparing the neural network classification approach with a random approach using the APFD metric.

## 6.1 Results for Experiments with *Partial Features*

**6.1.1 Results for the Real Dataset - *grep* Object.** The accuracy, precision and recall for the *grep* dataset using hold-out validation method are provided in Table 2. *Model 3* performs the best in terms of accuracy, and *Model 6* obtained best precision.

**Table 2: Accuracy, precision and recall results for every model using hold-out validation method for the *grep* dataset. Experiments with *partial features*.**

| Model no. | Accuracy     | Precision    | Recall |
|-----------|--------------|--------------|--------|
| 1         | 51.06        | 41.05        | 82.97  |
| 2         | 42.55        | 44.92        | 65.95  |
| 3         | <b>57.44</b> | 44.68        | 89.36  |
| 4         | 40.42        | 42.37        | 53.19  |
| 5         | 53.19        | 47.72        | 89.36  |
| 6         | 55.31        | <b>50.81</b> | 65.95  |

**6.1.2 Results for the Synthetic Dataset - *grep* - Based Generated Samples.** The generated synthetic dataset starting from the *grep* dataset was also used in our experiments (with extra 500 samples). We used both hold-out validation and k-fold validation (k=2) to conduct the experiments for the newly constructed dataset.

### Holdout validation

Table 3 reports the accuracy, precision and recall for the synthetic dataset using the hold-out validation. *Model 3* performs the best in terms of accuracy (47.42), and *Model 4* obtained best precision (41.34).

**Table 3: Accuracy, precision and recall results for every model using hold-out validation method for *grep*-based synthetic generated dataset. Experiments with *partial features*.**

| Model no. | Accuracy     | Precision    | Recall |
|-----------|--------------|--------------|--------|
| 1         | 40.20        | 38.19        | 78.35  |
| 2         | 44.32        | 40.21        | 76.28  |
| 3         | <b>47.42</b> | 37.37        | 79.38  |
| 4         | 46.39        | <b>41.34</b> | 76.28  |
| 5         | 30.92        | 37.36        | 73.19  |
| 6         | 40.20        | 38.99        | 63.91  |

### K-fold validation

For the *grep*-based synthetic generated dataset we have also used k-fold validation (k=2). Table 4 outlines the accuracy: *Model 6* performs the best in terms of accuracy (44.53) for Fold 1. We selected k=2 to have two more experiments similar in number of samples as the *grep* object dataset.

## 6.2 Results for Experiments with *All Features*

**6.2.1 Results for the Real Dataset - *grep* Object.** The accuracy, precision and recall for the *grep* dataset using hold-out validation method are provided in Table 5. *Model 6* performs the best in terms of accuracy and precision. Also, *Model 1*, *Model 4* and *Model 5* obtained best recall.

**Table 4: Accuracy for every model using k-fold validation method for *grep*-based synthetic generated dataset. Experiments with *partial features*.**

| Model no. | Fold | Accuracy     |
|-----------|------|--------------|
| 1         | 1    | 40.20        |
|           | 2    | 41.23        |
| 2         | 1    | 43.91        |
|           | 2    | 39.58        |
| 3         | 1    | 36.70        |
|           | 2    | 43.29        |
| 4         | 1    | 42.68        |
|           | 2    | 41.85        |
| 5         | 1    | 39.79        |
|           | 2    | 38.6         |
| 6         | 1    | <b>44.53</b> |
|           | 2    | 41.03        |

**Table 5: Accuracy, precision and recall results for every model using hold-out validation method for the *grep* dataset. Experiments with *all features*.**

| Model no. | Accuracy     | Precision | Recall |
|-----------|--------------|-----------|--------|
| 1         | 89.36        | 53.65     | 93.61  |
| 2         | <b>97.87</b> | 93.99     | 100.0  |
| 3         | 93.61        | 67.16     | 95.74  |
| 4         | <b>97.87</b> | 83.92     | 100.00 |
| 5         | 91.48        | 84.31     | 91.48  |
| 6         | <b>97.87</b> | 92.15     | 100.00 |

**6.2.2 Results for the Synthetic Dataset - *grep* - Based Generated Samples.** The generated synthetic dataset starting from the *grep* dataset was also used in our experiments (with extra 500 samples). We used both hold-out validation and k-fold validation (k=2) to conduct the experiments for the newly constructed dataset.

### Holdout validation

Table 6 reports the accuracy, precision and recall for the synthetic dataset using the hold-out validation. The best model is *Model 4* in term of accuracy and *Model 6* in term of precision.

**Table 6: Accuracy, precision and recall results for every model using hold-out validation method for *grep*-based synthetic generated dataset. Experiments with *all features*.**

| Model no. | Accuracy     | Precision | Recall |
|-----------|--------------|-----------|--------|
| 1         | 92.78        | 88.57     | 95.87  |
| 2         | <b>95.87</b> | 69.06     | 98.96  |
| 3         | 94.84        | 65.10     | 100.00 |
| 4         | <b>97.93</b> | 73.84     | 98.96  |
| 5         | 95.87        | 81.51     | 100.00 |
| 6         | <b>100</b>   | 67.36     | 100.00 |

Investigating the above results it is clear that the models that use the Adam optimizer obtained the best results.

### K-fold validation

For the *grep*-based synthetic generated dataset we have also used k-fold validation (k=2). Table 7 outlines the accuracy: *Model 6* performs the best in terms of accuracy (98.96) for Fold 2. We selected k=2 to have two more experiments similar in number of samples as the *grep* object dataset.

**Table 7: Accuracy for every model using k-fold validation method for *grep*-based synthetic generated dataset.**

| Model no. | Fold | Accuracy     |
|-----------|------|--------------|
| 1         | 1    | 92.57        |
|           | 2    | 91.95        |
| 2         | 1    | 97.93        |
|           | 2    | 97.31        |
| 3         | 1    | 91.54        |
|           | 2    | 92.16        |
| 4         | 1    | 96.90        |
|           | 2    | 98.35        |
| 5         | 1    | 89.69        |
|           | 2    | 88.86        |
| 6         | 1    | 97.73        |
|           | 2    | <b>98.96</b> |

## 6.3 Results for Statistical Tests

This section outlines the results of the conducted statistical tests comparing models with partial features with models with all the features, models with same architecture but different optimizers, and models with same optimizers but different architectures.

**6.3.1 Statistical Tests - Partial Features vs All Features.** The Wilcoxon Signed Rank Test was used to compare the proposed models with different number of used features. Table 8 contains the p-values. The obtained values are  $< 0.05$  for *Model 2*, *Model 3* and *Model 6*, i.e., we can reject  $H_0$ , thus there is a statistical difference between the models that used only partial features and the models that used all features (for the corresponding used architectures).

**Table 8: Wilcoxon Signed Rank test's results for holdout validation method Partial features vs All features.**

| Models p-values                  | <i>grep</i>  |
|----------------------------------|--------------|
| Model 1: Partial vs All Features | 0.463        |
| Model 2: Partial vs All Features | <b>0.010</b> |
| Model 3: Partial vs All Features | <b>0.003</b> |
| Model 4: Partial vs All Features | 0.578        |
| Model 5: Partial vs All Features | 0.109        |
| Model 6: Partial vs All Features | <b>0.017</b> |

**6.3.2 Statistical Tests - sgb vs Adam Optimizer - with All Features.** The Wilcoxon Signed Rank Test was used to compare the proposed models, either varying the optimizer used (sgb vs Adam), or the architecture of the NN (comparisons between all architectures using

the sgb optimizer, respectively comparisons between all architectures using the Adam optimizer). Table 9, Table 10 and Table 11 contain the p-values. The obtained values are not  $< 0.5$ , i.e., we can not reject  $H_0$ , thus we can not make any conclusions on the obtained results (on the differences of the proposed models).

**Table 9: Wilcoxon Signed Rank test's results for holdout validation method (different optimizers)**

| Models p-values    | <i>grep</i> | <i>grep</i> -based synthetic |
|--------------------|-------------|------------------------------|
| Model 1 vs Model 2 | 0.527       | 0.739                        |
| Model 3 vs Model 4 | 0.317       | 0.157                        |
| Model 5 vs Model 6 | 0.414       | 0.257                        |

**Table 10: Wilcoxon Signed Rank test's results for holdout validation method (sgb optimizer) - various architectures**

| Models p-values    | <i>grep</i> | <i>grep</i> -based synthetic |
|--------------------|-------------|------------------------------|
| Model 1 vs Model 3 | 1.0         | 0.366                        |
| Model 1 vs Model 5 | 0.096       | 0.564                        |
| Model 3 vs Model 5 | 0.705       | 0.655                        |

**Table 11: Wilcoxon Signed Rank test's results for holdout validation method (Adam optimizer) - various architectures**

| Models p-values    | <i>grep</i> | <i>grep</i> -based synthetic |
|--------------------|-------------|------------------------------|
| Model 2 vs Model 4 | 1.0         | 1.0                          |
| Model 2 vs Model 6 | 1.0         | 0.564                        |
| Model 4 vs Model 6 | 0.317       | 0.317                        |

As a result of the performed statistical test we can conclude that there is a difference between the models with partial features and models with all features when the number of hidden neurons increases (40 to 60) in the case of the Adam optimizer. The number of layers seems also to be important for the Adam optimizer.

## 6.4 Test Case Prioritization Comparison: NN-TCP versus Random-TCP

We have generated a new synthetic dataset of 470 test cases.

We generated a random permutation of the 470 test cases and calculated the APFD metric: 16%.

We have applied Model 6 for the same generated dataset, order the 470 test cases according to the classification obtained, and we obtained the APFD metrics: 18%.

Thus, our NN classification-based TCP approach obtained a better fault detection capability.

## 7 THREATS TO VALIDITY

It is not unusual for experimental analyzes to suffer from some threats to validity that can affect the results of the study; therefore, our test case prioritization approach is no exception to this rule. This section discusses a number of factors that may have influenced the results obtained.

*Threats to internal validity* refer to the subjectivity introduced in selecting the various parameters or metrics for the models. During model construction, we split the dataset between training and testing. To minimize threats to internal validity, a stratified split was chosen to ensure that the train and test sets have approximately the same percentage of samples from each target class as the complete set. Regarding the training of the models, we tested a set of hyperparameter values and reported the values that performed best. However, the results could be improved with a more detailed hyper-parameter optimization. We leave this for future work. The selected metrics may also affect the outcome of the study. Most of them, such as accuracy, precision, or recall, have been chosen with consideration to previous machine learning researches; however, we employed them from a regression testing perspective.

*Threats to external validity* are related to the generalization of the results obtained. Only two projects were considered for evaluation. We tried to reduce threats to external validity by choosing projects of different sizes (real-world project grep with 470 test cases and one synthetic dataset with 470+500 samples). We plan to extend the experimental evaluation to other large-scale software systems, analyzing other projects, and also using more metrics that could influence the priority of the test cases to be executed.

## 8 CONCLUSION AND FUTURE WORK

The paper proposes an approach for test case prioritization by applying artificial neural networks. The association between requirements, tests and discovered faults is included as part of the factors based on which the neural network is able to assign priorities to new test cases. The neural network models are validated using two datasets: a real dataset and a synthetic one, generated based on the real dataset using the Gaussian Mixture Model. In these two cases, the reliability of the proposed models for the ANN classifier has been evaluated. The outcome of the results indicates a better reliability for Adam optimizer and ANN with a three-layer architecture. To compare various proposed models from various perspectives (NN architecture, optimizer, number of features used, used dataset, validation method), statistical tests were also used.

Future work will investigate the test case prioritization classification problem by combining other metrics as independent variables. We also plan to extend the experimental evaluation considering other open source projects in order to assess the efficiency of the proposed metrics for the target vector.

## ACKNOWLEDGMENTS

This work was partially funded by Babes-Bolyai University, student fellowships *Special Scholarship for Scientific Activity*, number 36968/17.12.2021.

## REFERENCES

- [1] 2017. Software-artifact Infrastructure Repository. <http://sir.unl.edu/>. Accessed: 2016-06-06.
- [2] Paul Ammann and Jeff Offutt. 2016. *Introduction to Software Testing* (2 ed.). Cambridge University Press. <https://doi.org/10.1017/9781316771273>
- [3] Joaquin Derrac, Salvador Garcia, Daniel Molina, and Francisco Herrera. 2011. A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms. *Swarm and Evolutionary Computation* 1, 1 (2011), 3–18. <https://doi.org/10.1016/j.swevo.2011.02.002>
- [4] T.L. Graves, M.J. Harrold, J. Kim, A. Porters, and G. Rothermel. 1998. An empirical study of regression test selection techniques. In *Proceedings of the 20th International Conference on Software Engineering*. 188–197. <https://doi.org/10.1109/ICSE.1998.671115>
- [5] Hosney Jahan, Ziliang Feng, S.M. Hasan Mahmud, and Penglin Dong. 2019. Version Specific Test Case Prioritization Approach Based on Artificial Neural Network. *J. Intell. Fuzzy Syst.* 36, 6 (jan 2019), 6181–6194. <https://doi.org/10.3233/JIFS-181998>
- [6] Barbara Kitchenham and Stuart Charters. 2007. Guidelines for performing Systematic Literature Reviews in Software Engineering. 2 (01 2007).
- [7] Remo Lachmann, Sandro Schulze, Manuel Niek, Christoph Seidl, and Ina Schaefer. 2016. System-Level Test Case Prioritization Using Machine Learning. In *2016 15th IEEE International Conference on Machine Learning and Applications (ICMLA)*. 361–368. <https://doi.org/10.1109/ICMLA.2016.0065>
- [8] Chu-Ti Lin, Kai-Wei Tang, and Gregory M. Kapfhammer. 2014. Test suite reduction methods that decrease regression testing costs by identifying irreplaceable tests. *Information and Software Technology* 56, 10 (2014), 1322–1344. <https://doi.org/10.1016/j.infsof.2014.04.013>
- [9] Mario A. Muñoz, Laura Villanova, Davaatseren Baatar, and Kate Smith-Miles. 2018. Instance Spaces for Machine Learning Classification. *Mach. Learn.* 107, 1 (jan 2018), 109–147. <https://doi.org/10.1007/s10994-017-5629-5>
- [10] G. Rothermel, R.H. Untch, Chengyun Chu, and M.J. Harrold. 1999. Test case prioritization: an empirical study. In *Proceedings IEEE International Conference on Software Maintenance - 1999 (ICSM'99)*. 'Software Maintenance for Business Change' (Cat. No.99CB36360). 179–188. <https://doi.org/10.1109/ICSM.1999.792604>
- [11] S. Russel and P. Norvig. 1995. *Artificial intelligence: a modern approach*. Englewood Cliffs, N.J.: Prentice Hall.
- [12] Yogesh Singh, Arvinder Kaur, and Bharti Suri. 2010. Test Case Prioritization Using Ant Colony Optimization. *SIGSOFT Softw. Eng. Notes* 35, 4 (jul 2010), 1–7. <https://doi.org/10.1145/1811226.1811238>
- [13] Manika Tyagi and Sona Malhotra. 2014. Test case prioritization using multi objective particle swarm optimizer. In *2014 International Conference on Signal Propagation and Computer Technology (ICSPCT 2014)*. 390–395. <https://doi.org/10.1109/ICSPCT.2014.6884931>
- [14] Andreea Vescan, Camelia Chisaliță-Cretu, Camelia Serban, and Laura Diosan. 2021. On the Use of Evolutionary Algorithms for Test Case Prioritization in Regression Testing Considering Requirements Dependencies. In *Proceedings of the 1st ACM International Workshop on AI and Software Testing/Analysis (Virtual, Denmark) (AISTA 2021)*. Association for Computing Machinery, New York, NY, USA, 1–8. <https://doi.org/10.1145/3464968.3468407>
- [15] Andreea Vescan, Camelia Șerban, Camelia Chisăliță-Cretu, and Laura Diosan. 2017. Requirement dependencies-based formal approach for test case prioritization in regression testing. In *2017 13th IEEE International Conference on Intelligent Computer Communication and Processing (ICCP)*. 181–188. <https://doi.org/10.1109/ICCP.2017.8117002>
- [16] Shin Yoo and Mark Harman. 2007. Pareto Efficient Multi-Objective Test Case Selection. In *Proceedings of the 2007 International Symposium on Software Testing and Analysis (London, United Kingdom) (ISSA '07)*. Association for Computing Machinery, New York, NY, USA, 140–150. <https://doi.org/10.1145/1273463.1273483>
- [17] Shin Yoo and Mark Harman. 2010. Using hybrid algorithm for Pareto efficient multi-objective test suite minimisation. *Journal of Systems and Software* 83, 4 (2010), 689–701. <https://doi.org/10.1016/j.jss.2009.11.706>